Software Architecture and Design Specification

Project: Online Bookstore

Version: 1.0

Authors:
Zia Kadijah (SRN: PES2UG24CS620)
Hithesh Nanjunda Swamy (SRN: PES2UG24CS635)
Nidhi Yedida (SRN: PES2UG24CS673)
Vidya Sri Ganesh (SRN: PES2UG24CS667)

Date: 08-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	08-10-2026	Project Team	Initial SAD draft

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Bookstore.

1.2 Scope

Covers Online Bookstore user services: account registration and login, catalog browsing and search, cart and wishlist, checkout and payment, order tracking, reviews and ratings, and seller/admin management.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

SRS, STP, ADR, API, JWT, ORM, RBAC, PCI-DSS, TLS, STRIDE, REST.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, ADRs, threat models, and API design.

2.2 Related Documents

Online Bookstore SRS v1.0, STP v1.0, RTM.

3. Architecture

3.1 Goals & Constraints

Goals: secure, reliable, 99.5% availability, catalog and product pages loading in 2 seconds or less.
Constraints: PCI-DSS compliance, mandatory TLS, third-party payment gateway rate limits.

3.2 Stakeholders & Concerns

- Buyers: security, usability, performance
- Sellers: easy inventory and order management
- Admins: oversight, auditability, control
- Payment/Regulators: compliance
- Developers: modularity, maintainability

3.3 Component (UML) Diagram

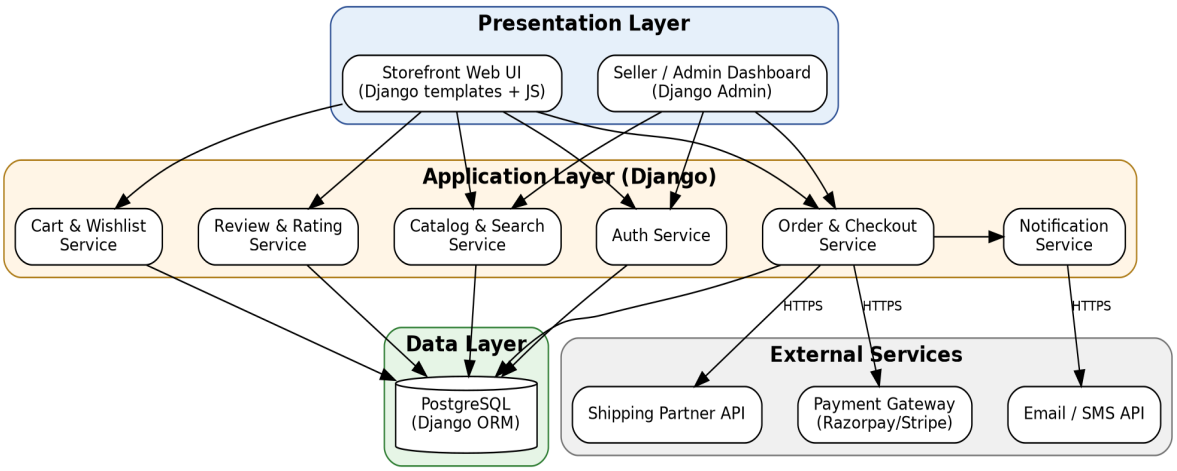


Figure 1: Component diagram of the Online Bookstore

3.4 Component Descriptions

- Storefront Web UI: handles buyer input/output (browse, cart, checkout).
- Seller/Admin Dashboard: manages listings, inventory, users, and orders (built on Django Admin).
- Auth Service: registers users, validates credentials, issues JWT, enforces RBAC and lockout.
- Catalog & Search Service: serves paginated catalog, search, filters, and sorting.
- Cart & Wishlist Service: manages cart and wishlist items per user.
- Order & Checkout Service: validates stock, creates orders, coordinates payment and shipping, tracks status.
- Review & Rating Service: stores and aggregates verified-buyer reviews.
- Notification Service: sends order and account emails/SMS asynchronously.
- Database: PostgreSQL storing users, books, orders, and reviews via the Django ORM.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture (presentation, application, data) built as a modular Django monolith, chosen for clear separation of concerns and fast development. Microservices rejected as overly complex for the project's scale.

3.6 Technology Stack & Data Stores

Python, Django and Django REST Framework APIs, PostgreSQL DB, JWT authentication, Celery with Redis for asynchronous notifications, TLS encryption. Payments via Razorpay/Stripe (tokenised).

3.7 Risks & Mitigations

- Risk: payment gateway downtime → Mitigation: retry with backoff and keep orders in a pending state.
- Risk: database failure or data loss → Mitigation: automated backups and a standby replica.
- Risk: traffic spikes slowing the storefront → Mitigation: caching and CDN for static assets.

3.8 Traceability to Requirements

- BOOK-F-001, BOOK-F-002 (registration, authentication) → Auth Service
- BOOK-F-010, BOOK-F-011 (catalog, search) → Catalog & Search Service
- BOOK-F-020 to 023 (cart, wishlist) → Cart & Wishlist Service
- BOOK-F-030 to 035, BOOK-F-040 to 044 (checkout, orders) → Order & Checkout Service
- BOOK-F-050 to 054 (seller/admin) → Seller/Admin Dashboard
- BOOK-F-060 to 062 (reviews) → Review & Rating Service
- BOOK-SR-003 (no card data stored) → Order & Checkout Service + Payment Gateway tokenisation

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → email verification, JWT, account lockout; Tampering → TLS, CSRF protection, ORM-based input handling; Repudiation → audit logging of admin actions; Info disclosure → TLS, salted password hashing, tokenised payments; DoS → rate limiting and CDN; Elevation of Privilege → RBAC.

4. Design

4.1 Design Overview

The Online Bookstore is designed with layered Django modules (one app per service) for separation of concerns and easier maintenance.

4.2 UML Sequence Diagrams

Two flows specific to this project: user login and checkout with payment.

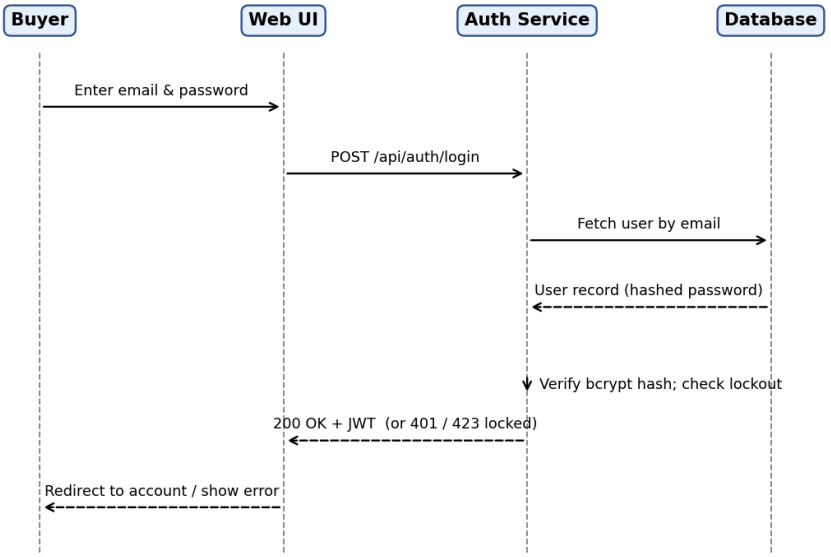


Figure 2: Sequence diagram – User login

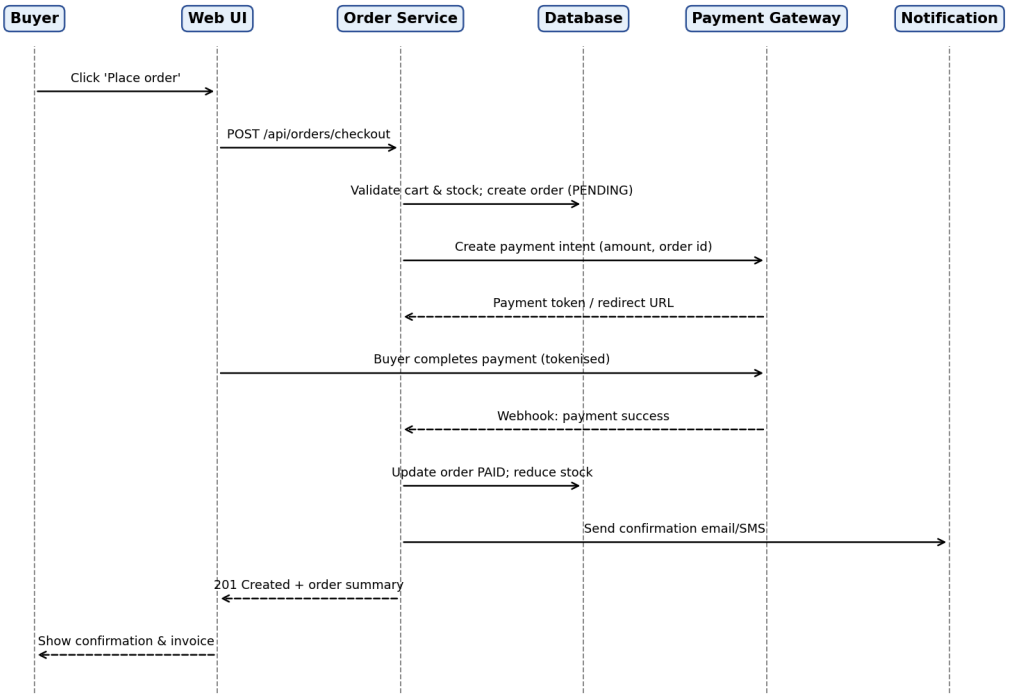


Figure 3: Sequence diagram – Checkout and payment

4.3 API Design

Auth Service

Field	Details
Endpoint	/api/auth/login
Method	POST
Request	{email, password}

Field	Details
Response	{status, token, role}
Errors	401 Invalid credentials, 423 Account locked

Catalog & Search Service

Field	Details
Endpoint	/api/books
Method	GET
Request	Query params: q, genre, sort, page
Response	{results[{id, title, author, price, cover}], page, total}
Errors	400 Invalid parameters

Order & Checkout Service

Field	Details
Endpoint	/api/orders/checkout
Method	POST
Request	{cartId, addressId, couponCode?}
Response	{orderId, status, paymentUrl}
Errors	401 Not authenticated, 409 Out of stock, 502 Payment gateway error

4.4 Error Handling, Logging & Monitoring

Standardized JSON error messages ({code, message}). No sensitive info (passwords, card data, tokens) in logs. Monitoring includes checkout failure rate, API response time, and system uptime.

4.5 UX Design

Responsive storefront with accessible font sizes and colour contrast (WCAG 2.1 AA), a prominent search bar, guest browsing, and a clear step-by-step checkout flow.

4.6 Open Issues & Next Steps

Future enhancements: personalised book recommendations, wishlist sharing, multi-language support, e-book and audiobook formats, mobile app.

5. Appendices

5.1 Glossary: ADR, API, JWT, ORM, RBAC, PCI-DSS, TLS, STRIDE.

5.2 References: IEEE 42010, OWASP, PCI-DSS, Django documentation.

5.3 Tools: Graphviz, draw.io, Swagger/OpenAPI, Postman.